[CSED233] Data Structure

# List, Stack, and Queue

Seung-Hwan Baek

POSTECH

# Review: Quiz on C++

- Array, pointer, dynamic memory

```
char c[] = {'c', 'a', 't'};
char* p = c;
char* q = &c[0];
cout << c[2] << p[2] << q[2];
```

# Review: Quiz on Big O

- Which algorithm is better in terms of Big O?

$$O(2^n)$$

$$\text{vs}$$

$$O(n^2)$$

# Review: Quiz on Big O

- Any algorithm with the factorial time complexity?

# List

- List $L = < a_1, a_2, \ldots, a_n >$
  - a finite, ordered collection of elements
  - $n$: length (size) of the list
    - empty list <> : $n = 0$ (no elements)

- Position of $a_i$ is $i$
  - head (front) $\leftrightarrow$ tail (rear)
  - "current" position

$$\begin{array}{ccc} \text{head} & & \text{tail} \\ \downarrow & & \downarrow \\ < a_1, & a_2, \ldots, & a_n > \end{array}$$

$< 20,\ 23, \mid 12,\ 15 >$ : separated by fence

$< 20,\ 23, \mid 10,\ 12,\ 15 >$ after insertion of $10$ (at "current" position)

# Quiz

$$< 20, \quad 23, \quad 12, \quad 15 >$$

- Is this list is sorted?

# Quiz

$$< 20, \quad 23, \quad 12, \quad 15 >$$

- Is this list is ordered?

# Sorted vs. Ordered

- Don't be confused, ordered and sorted mean different things
- Sorted list

$$< 12, \quad 15, \quad 20, \quad 23 >$$

- Ordered list

$$< 20, \quad 23, \quad 12, \quad 15 >$$
$$< 12, \quad 15, \quad 20, \quad 23 >$$

# List Operations

- $L = < a_1, a_2, \dots, a_{p-1}, a_p, a_{p+1}, \dots, a_n >$
  - $Insert(x, p, L)$ : insert $x$ at position $p$ in list $L$
    - Result: $< a_1, a_2, \dots, a_{p-1}, x, a_p, \dots, a_n >$
  - $Delete(p, L)$ : delete element at position $p$ in $L$
    - Result: $< a_1, a_2, \dots, a_{p-1}, a_{p+1}, \dots, a_n >$
  - $Next(p, L)$ : returns the position or pointer immediately following position $p$
  - *Previous(p, L)* : returns the position or pointer previous to $p$
  - *Locate(x, L)* : returns the position or pointer of $x$ on $L$
  - *Retrieve(p, L)* : returns element at position $p$ on $L$
  - *MakeNull(L)* : causes $L$ to become an empty list and returns position *END(L)*
  - *First(L)* : the first position on $L$

* This is just one example. There is no rule about the function specifications

# Array-Based List Implementation (1)

- $L = < a, b, c, d >$
  - $char\ L[\max]$

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $a$ | $b$ | $c$ | $d$ | | | | | | | | | | | |
| $1$ | $2$ | $3$ | $4$ | $5$ | $\bullet\bullet\bullet$ | | | | | | | | | $max$ |

  - Integer $size = 4$ or
    - Integer $last = the\ position\ of\ the\ last\ element$

- $Insert\ (x, 2, L)$ ?

# Array-Based List Implementation (2)

- $L = < a, b, c, d >$
  - $char\ L[\max]$

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $a$ | $b$ | $c$ | $d$ | | | | | | | | | | | |
| $1$ | $2$ | $3$ | $4$ | $5$ | $\bullet\bullet\bullet$ | | | | | | | | | $max$ |

  - Integer $size = 4$ or
    - Integer $last = the\ position\ of\ the\ last\ element$

- $Insert\ (x, 2, L)$ ?

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $a$ | | $b$ | $c$ | $d$ | | | | | | | | | | |
| $1$ | $2$ | $3$ | $4$ | $5$ | $\bullet\bullet\bullet$ | | | | | | | | | $max$ |

# Array-Based List Implementation (3)

- $Insert\ (x, 2, L)$
  - $void\ insert(char\ x, int\ i, char\ *\ L)$

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $a$ | $x$ | $b$ | $c$ | $d$ | | | | | | | | | | |
| 1 | 2 | 3 | 4 | 5 | ••• | | | | | | | | | max |

  - Before insertion, we need to *shift right* all following elements by one position
  - $size\ =\ 5$
  - Running time? $O(n)$ – Linear time

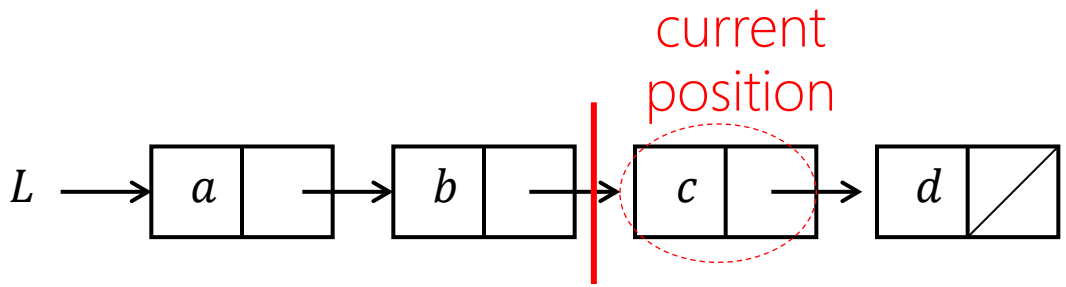
- $Delete(p, L), Locate(x, L)$
  - Running time? $O(n)$ – Linear time

# Pointer-Based List Implementation (1)

- Singly-linked list (One-way list)

$$L = < a, b, | c, d >$$

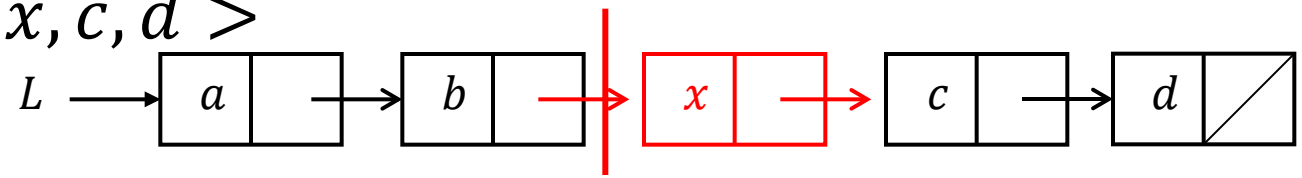
```
Struct item{
  char info;
  item * link;
};
```




- Node (or cell) = **info** + *link*

- $Insert\ (x, p, L)$ when $p = current$

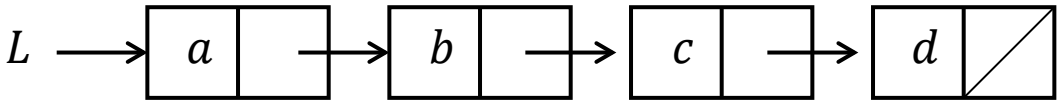
$$L = < a, b, | x, c, d >$$



# Quiz

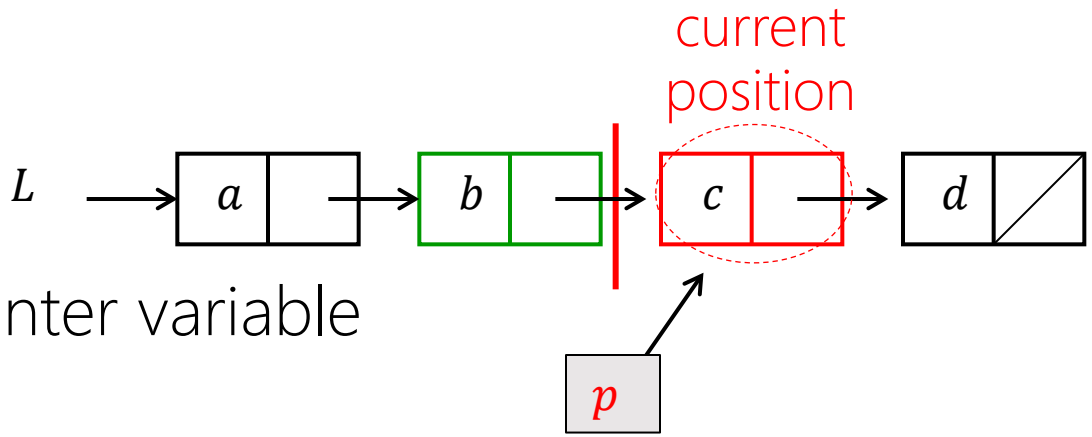
- How can we change the singly-linked list to the doubly-linked list?

```
Struct item{
  char info;
  item * link;
};
```



# Pointer-Based List Implementation (2)

- How to represent the logical fence?

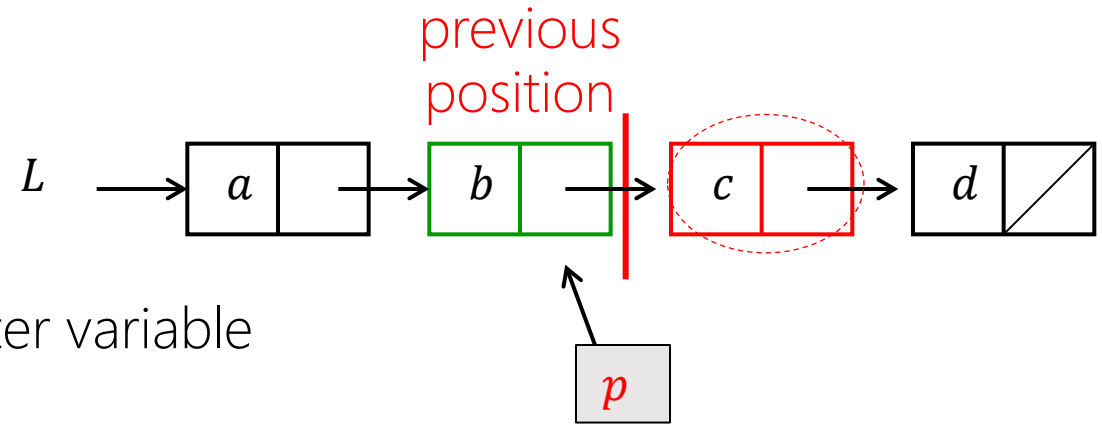



  - We need a pointer variable

- Option-1:
  - $p$ directly points to the current element
  - What difficulty for $Insert\ (x, p, L)$ ?
    - Inconvenient access to the preceding node of the current one
    - We have to change the link of the preceding node

# Pointer-Based List Implementation (3)

- How to represent the logical fence?

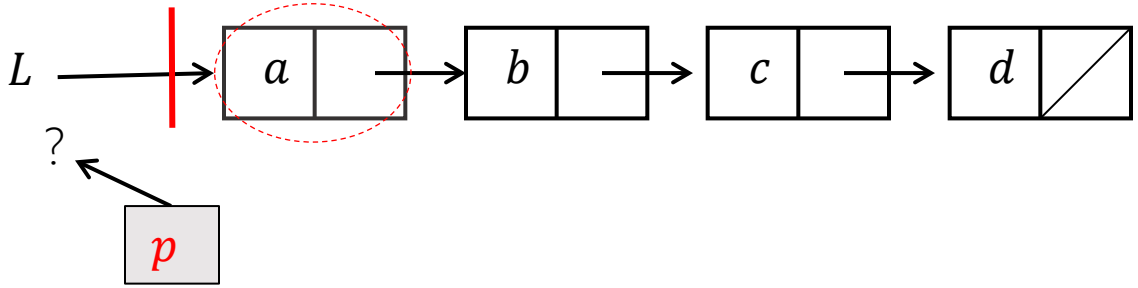



  - We need a pointer variable

- Option-2 (One-step ahead convention):
  - $p$ points to the previous element of the current position
  - Different definition of position
    - $p$ is the position of the element "$b$" (not "$c$")

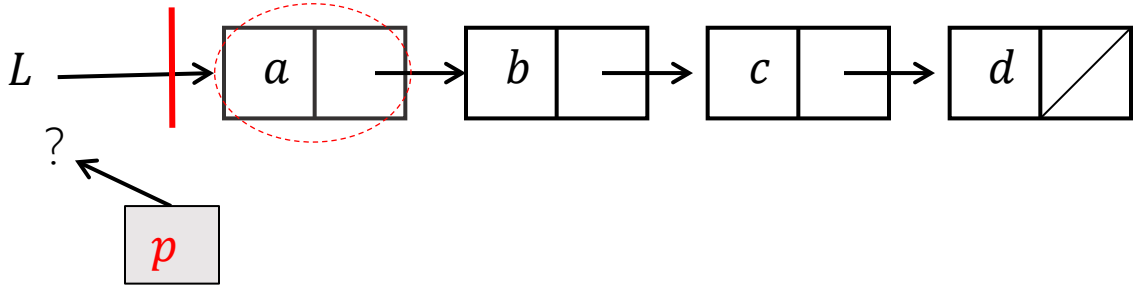
# Pointer-Based List Implementation (4)

- $Insert\ (x, p, L)$
  - When the list is empty or the left partition is empty, what happens?



# Pointer-Based List Implementation (4)

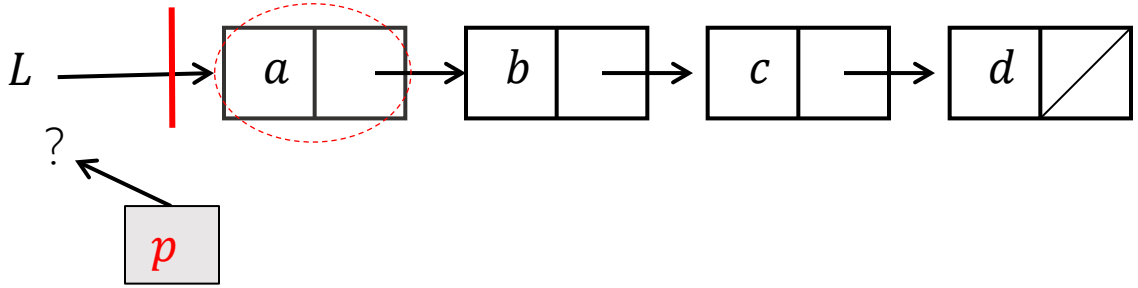
- $Insert\ (x, p, L)$
  - When the list is empty or the left partition is empty, what happens?



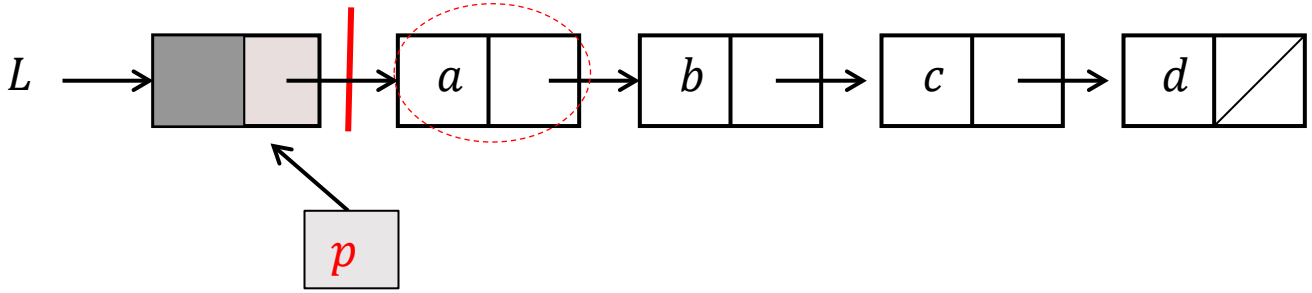

- Solution 1: implement special case handling using "if" statement

# Pointer-Based List Implementation (4)

- $Insert\ (x, p, L)$
  - When the list is empty or the left partition is empty, what happens?

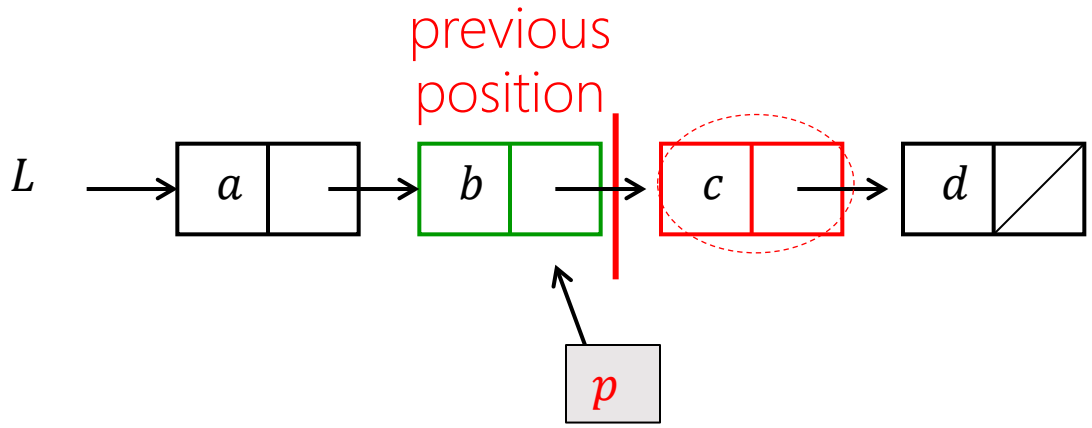



- Solution 1: implement special case handling using "if" statement
- Solution 2: (dummy) header node



# Quiz

- What is the time complexity of $Insert\ (x, p, L)$?



# Cursor-Based List Implementation

- Cursor (Simulated Pointer)
  - Integer index indicating positions in array to simulate pointers
  - We maintain the heads of the lists
  - Zero value of the link means the tail of the list

- *Example*
  - $L_1 = < a, b, c >$ $\quad L_2 = < x, y >$
- Time complexity of insert and delete
  - $O(1)$
- Time complexity of retrieve
  - $O(n)$

| | info | *link* |
|---|---|---|
| 1 | $c$ | *0* |
| 2 | $a$ | *4* |
| 3 | $y$ | *0* |
| 4 | $b$ | *1* |
| 5 | | |
| 6 | $x$ | *3* |
| 7 | ... | |

*Head of* $L_1$ → 2

*Head of* $L_2$ → 6

# Stacks

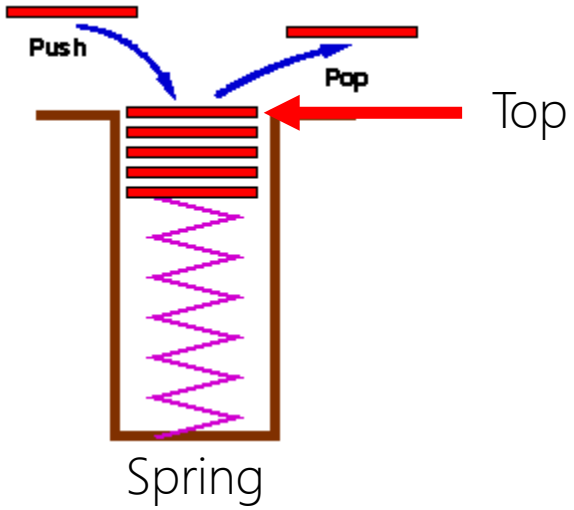
- All insertions & deletions take place at one end (called Top)
- Special type of list, often called as LIFO (Last-In, First-Out)
- You can implement stacks using any type of list implementation (pointer, array, cursor, …)

- $S = < a_1, a_2, \dots, a_n >$

  $\uparrow$

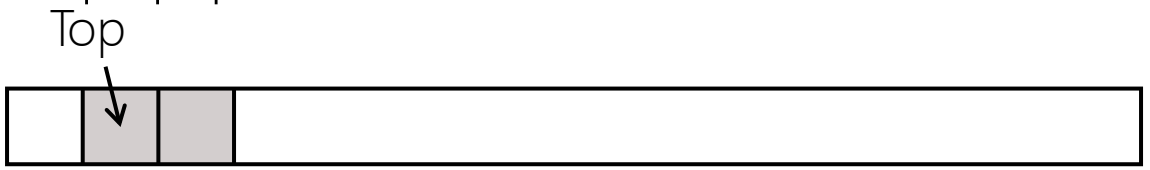
  Top

  - $Push(x, S)$ $\quad S = < x, a_1, a_2, \dots, a_n >$
  - $Pop(S)$ $\quad S = < a_2, \dots, a_n >$
  - $Top(S)$ $\quad a_1$
  - $MakeNull(S)$ $\quad S = <>$
  - $IsEmpty(S)$ $\quad true\ if\ S = <>$

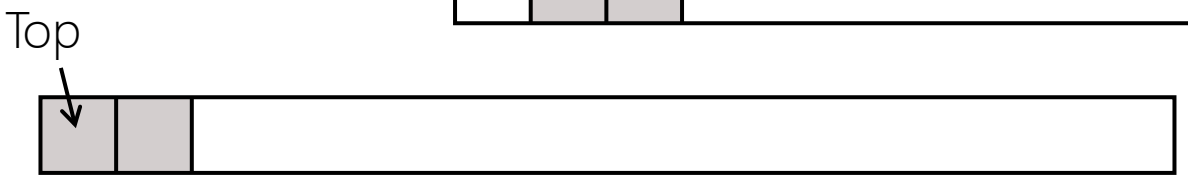


# Stack Implementations

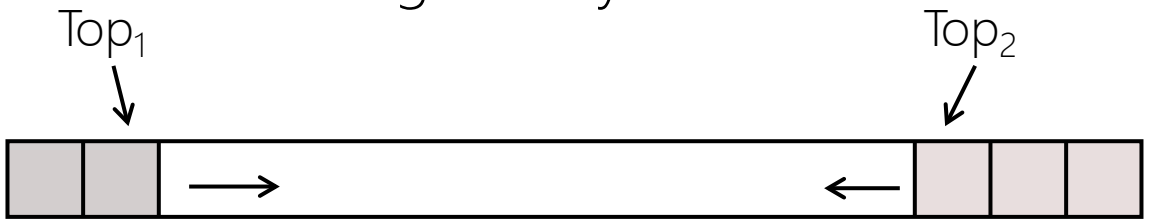
- Array-based stacks
  - How to implement TOP ? (in terms of cost of pop/push)
    - Position $k$ (when $k$ elements in stack): $O(1)$




    - (Fixed) Position $1$: $O(n)$




  - We can even store multiple stacks in a single array




- Linked stacks
  - Very much similar to the pointer-based list implementation

- Example: call stack

# Queue

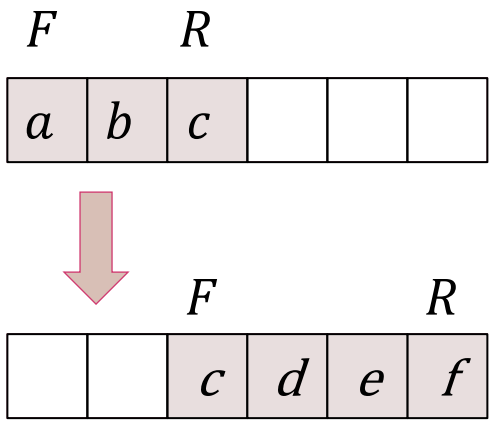
- FIFO (First-In First-Out) list
- Similar to "top" in the stack, here we have front and rear
- $Q = < a_1, a_2, \dots, a_n >$

  $\uparrow$ Front $\quad$ $\uparrow$ Rear

- $Enqueue(x, Q)$ $\quad$ $Q = < a_1, a_2, \dots, a_n, x >$
- $Dequeue(Q)$ $\quad$ $Q = < a_2, a_3, \dots, a_n >$
- $Front(Q)$ $\quad$ $a_1$
- $MakeNull(Q)$ $\quad$ $Q = <>$
- $IsEmpty(Q)$ $\quad$ $true\ if\ Q = <>$

# Array-Based Queues (1)

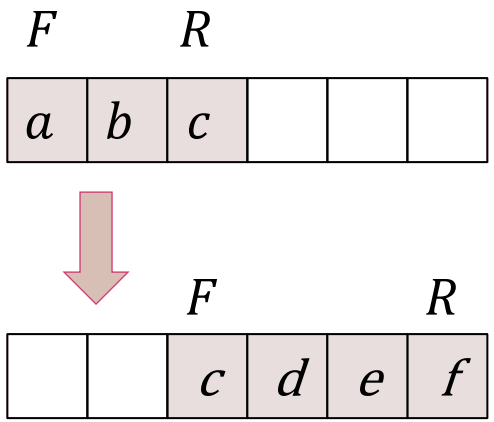
- We have an array of size 6
- You enqueue data "a", "b", "c"
- Dequeue two times
- Enqueue "d", "e", "f"




- Can we enqueue more data?

# Array-Based Queues (1)

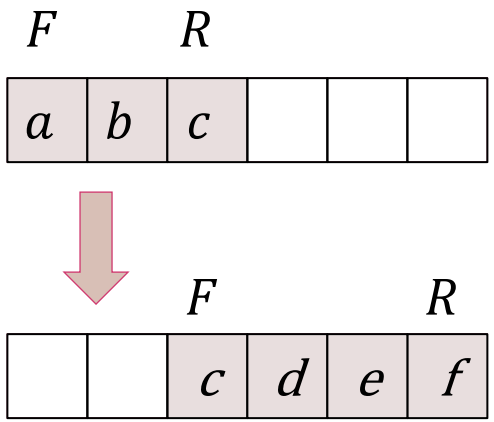
- We have an array of size 6
- You enqueue data "a", "b", "c"
- Dequeue two times
- Enqueue "d", "e", "f"




- Can we enqueue more data?
- This problem is called as "drifting queue" problem

# Quiz

- Without changing data structure, how can we enqueue new data?

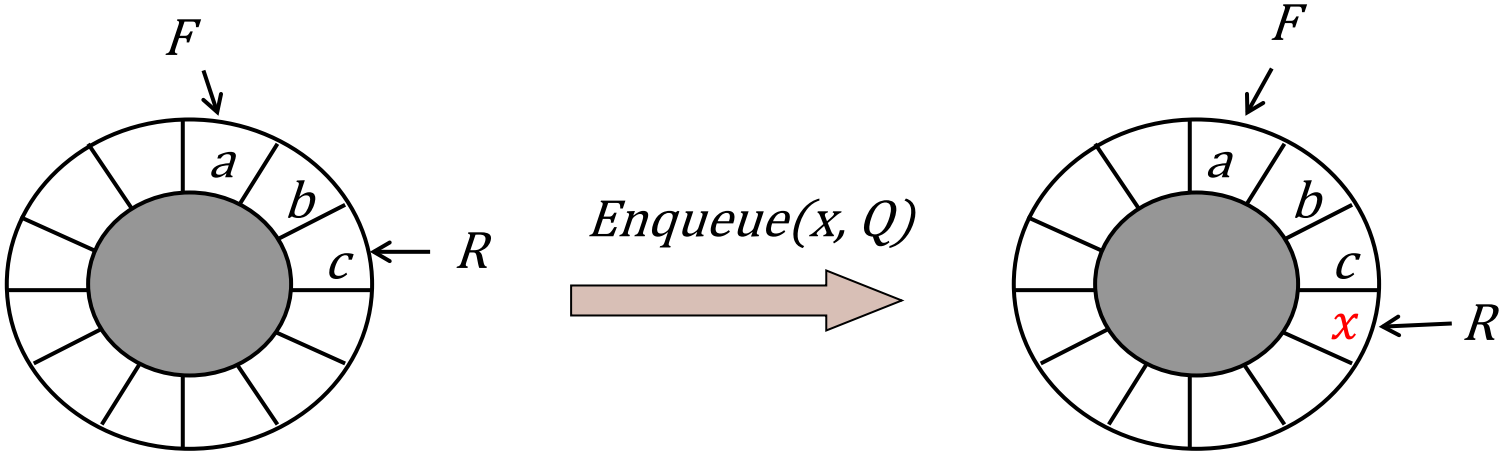



- In that case, what are the time complexities for enqueue and dequeue?

# Array-Based Queues (1)

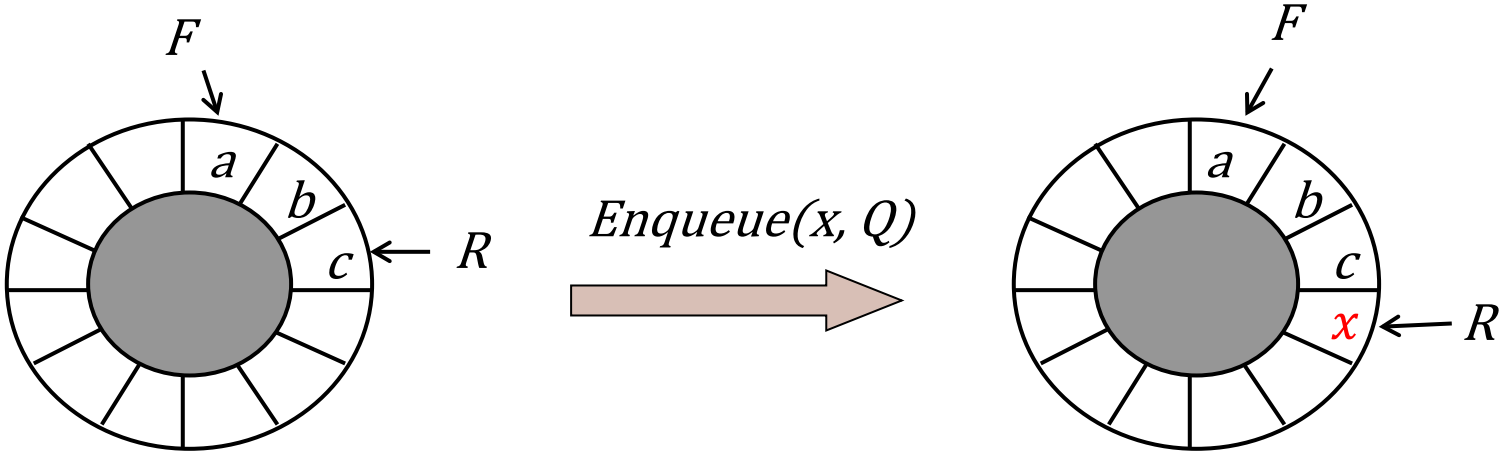
- Better data structure: circular array

- Utilize modulo operator: $n + 1 \equiv 1 \ (mod\ n)$
- Modulo operator allows us to mathematically connect the last element to the first element



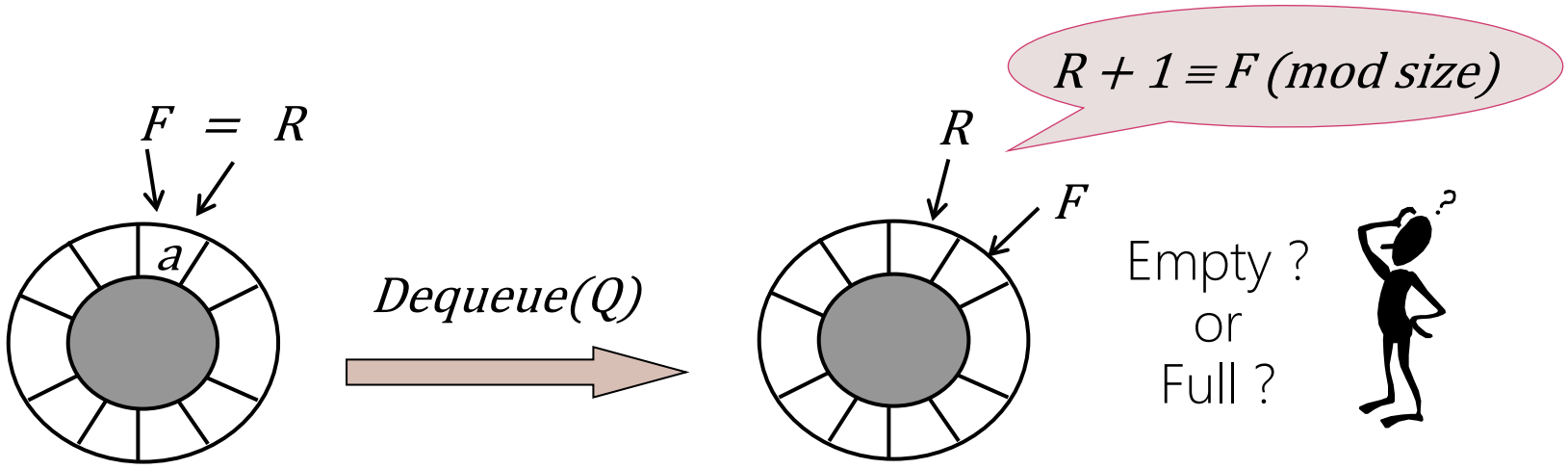
# Quiz

- How can we implement the enqueue and dequeue for the circular queue?



# Array-Based Queues (2)

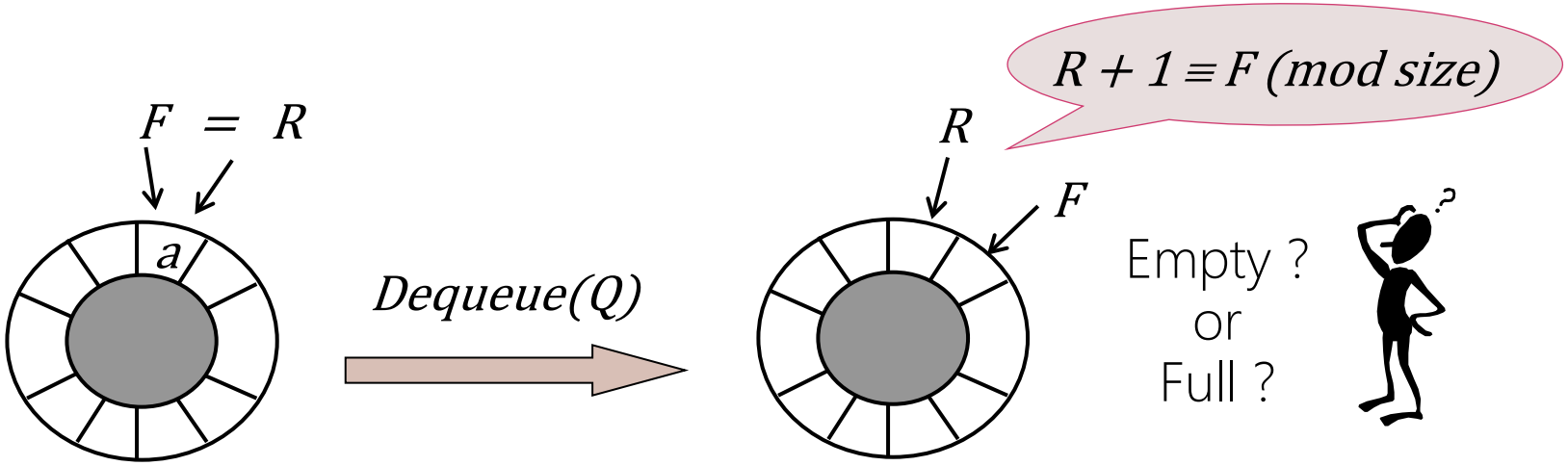
- How to recognize empty queue for the following example?




- Problem - indistinguishable from full queue

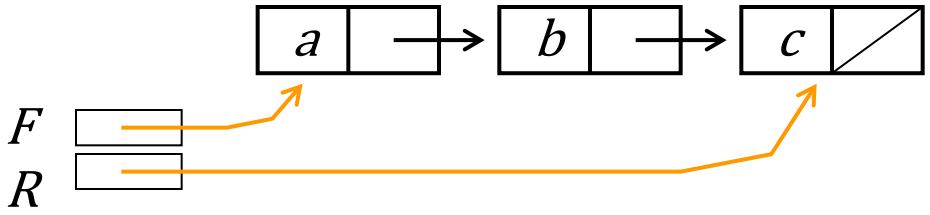
# Solutions?

- Explicit count variable (# of elements in queue)
  - For the enqueue: ***cnt ++;***
  - For the dequeue: ***cnt --;***
- Boolean variable (to indicating empty queue)
  - **bool *isempty;***



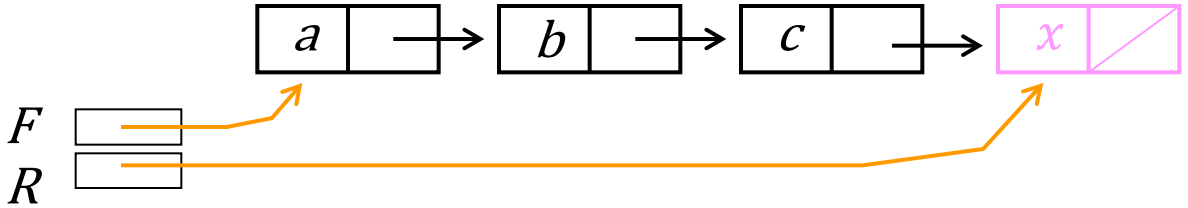
# Linked Queues (1)

- Two pointers $(F, R)$ (without a dummy header)




- $Empty(Q)$
  - $true$ if $(F = R \rightarrow null)$

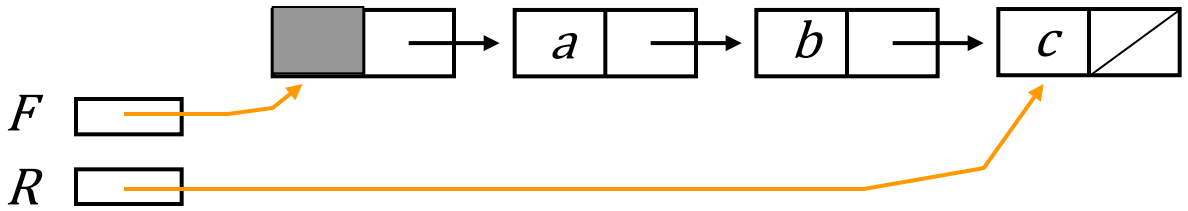
- $Enqueue(x, Q)$



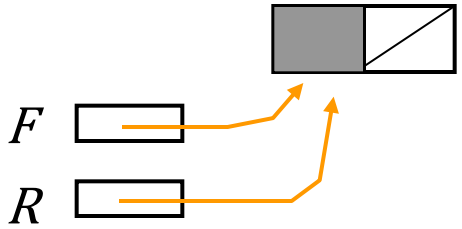

- Can we use the same code when inserting into an empty queue?

# Linked Queues (2)

- With a dummy header




- $Empty(Q)$
  - $true$ if $(F = R \rightarrow header)$



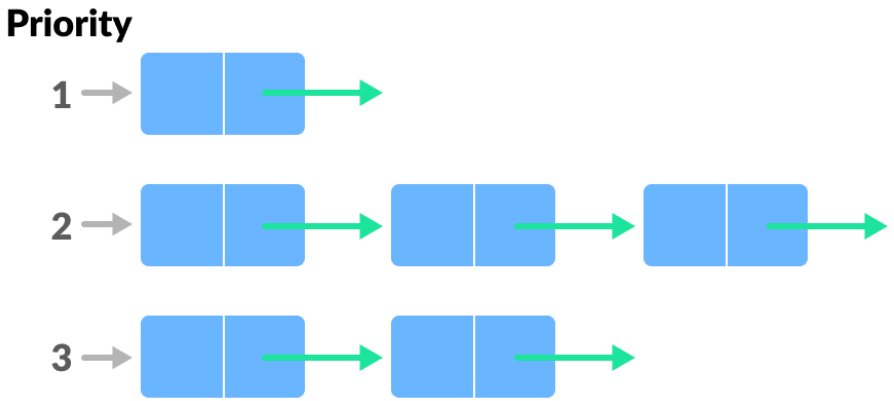

- $Enqueue(x, Q)$

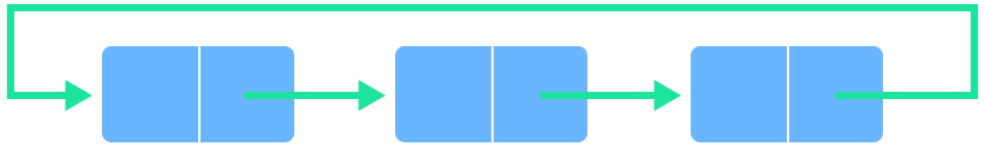
# Quiz: with vs. without dummy header

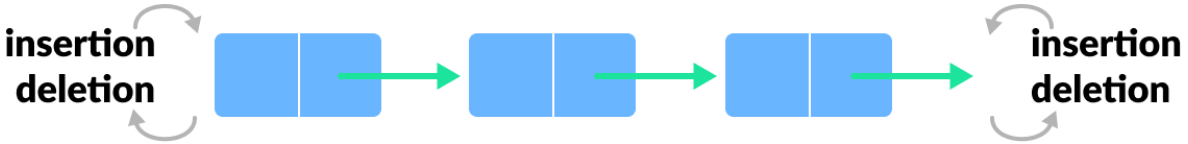
- Speed?
- Space utilization?
- Code conciseness?

# Type of Queues

- Queue, circular queue, priority queue, double-ended queue





# Code Testing

- Making test cases is one of the best ways to understand data structures
  - What cases will cause certain implementations to slow down?
  - How long do I expect certain operations to take?
  - What edge cases are there in the definition?
  - Where else might I find bugs?

Adopted from Robbie Weber

# Testing

- Software Test:
  a separate piece of code that exercises the code you are assessing by providing input to your code and finishes with an assertion of what the result should be.

1. Isolate -break your code into small modules
2. Build in increments -Make a plan from simplest to most complex cases
3. Test as you go -As your code grows, so should your tests

Adopted from Robbie Weber

# Types of Tests

- Black Box
  - -Behavior only –requirements
  - -From an outside point of view
  - -Does your code uphold its contracts with its users?
  - -Performance/efficiency

- White Box
  - -Includes an understanding of the implementation
  - -Written by the author as they develop their code
  - -Break apart requirements into smaller steps
  - -"unit tests" break implementation into single assertions

Adopted from Robbie Weber

# What to Test?

- Expected behavior
  - -The main use case scenario
  - -Does your code do what it should given friendly conditions?
- Forbidden Input
  - -What are all the ways the user can mess up?
- Empty/Null
  - -Protect yourself!
  - -How do things get started?
  - -0, -1, null, empty collections
- Boundary/Edge Cases
  - -First items
  - -Last item
  - -Full collections (resizing)
- Scale
  - -Is there a difference between 10, 100, 1000, 10000 items?

Slides from Robbie Weber

# Summary

- List
- List implementations
- Stack
- Stack implementations
- Queue
- Queue implementations
- Code testing